Basic Linear Algebra (융합기초선형대수학)

KNU Math 902

Classnotes

Mark Siggers

These notes are supplemental to a class taught from Trefethen and Bau's 'Numerial Linear Algebra'. It is expected that students have this text. It is herein referred to as 'the text'.

v. November 13, 2026

The text is on Numerical Linear Algebra, and supposes a familiarity with classical linear algebra. It expects that you know what matrices and vectors are, how matrix equations are a way of writing systems of linear equations, and how we can solve them using Gaussian Elimination. You should be familiar eigenvalues and eigenvectors of a matrix, have some notion that they are important, and have seen how to find them algebraically, even if you don't remember this completely.

It doesn't matter so much if you can remember it, because in practice, we aren't going to use Gaussian Elimination. I practice, we are using big matrices, and solving them with computers, so are generally not solving them algebraically.

Gaussian Elimination is a great theoretical tool for understanding the theory of matrices, for diagonalising matrices or finding their nullspace. We will 'recall' such things, but only briefly.

We will spend much more of our time on matrix decompositions and approximations. We will look first at the QR-decomposition of a matrix, and because matrices are seldom square, we will talk more of the single value decomposition than diagonalisation.

Our focus will be on not the simplest algorithms for working with matrices, but on those that work best on computers– stable algorithms, which control their error. It is a different game than classical linear algebra, but it has it own beauty.

Lecture 1 Matrix Vector Multiplication

Sep. 04

In this book, we work over the complex numbers $\mathbb{C}$, but it is usually okay to think of working over the reals $\mathbb{R}$. Probably all pictures, and much intuition, will be of the real case.

An $m \times n$ *matrix* $A = A_{m\times n}$ over the complex numbers $\mathbb{C}$ is a 'rectangle' of m rows and n columns of numbers from $\mathbb{C}$.

$$A = \begin{bmatrix} 1 & 2 & \frac{1}{2} \\ 4 & i & \pi \end{bmatrix}$$

is a 2×3 matrix. We write $A = [a_{i,j}]$ to designate that $a_{i,j}$ is the entry in the ith row (from the top) and the jth column. We drop the comma and write a_{ij} when it does not cause confusion. The values m and n are the *dimensions* of the matrix. A $1 \times n$ matrix is a *(column) vector* and a $m \times 1$ matrix is a *(row) vector*. When $A = [a_{ij}]$ we write a_{i*} for the ith row vector and a_{*j} for the jth column vector. The '$*$' runs over all values.

Vectors in $\mathbb{C}^m$ are viewed sometimes as points in $\mathbb{C}^m$, and sometimes as the 'arrow' from the origin $0 = (0, 0, \ldots, 0)$ to the point.

Practice

Where $A = [a_{ij}]$ is the 2×3 matrix $A = \left[\begin{smallmatrix} 1 & 2 & \frac{1}{2} \\ 4 & 0 & \pi \end{smallmatrix}\right]$, what is a_{23}? What is a_{*2}?

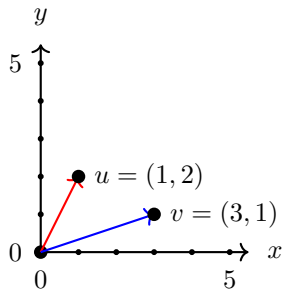
We add matrices of the same dimensions componentwise

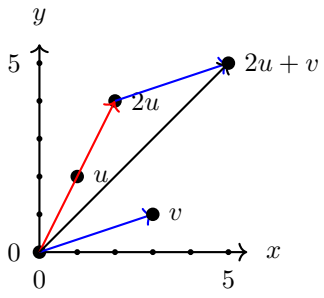
$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 2 \\ 2 & 3 & 4 \end{bmatrix}$$

and multiply matrices by *scalars* $r \in \mathbb{R}$ componentwise

$$2\begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix} = \begin{bmatrix} 2 & 2 & 2 \\ 2 & 4 & 6 \end{bmatrix}.$$

For vectors (either $m \times 1$ or $1 \times n$ matrices) there is an obvious interpretation of this.





We can also multiply matrices, but only if their dimensions are right. Where $A = A_{\ell\times m} = [a_{ij}]$ and $B = B_{m\times n} = [b_{jk}]$ the product $C_{\ell\times n} = AB$ is the matrix $C = [c_{ik}]$ where

$$c_{ik} = \sum_{j=1}^{m} a_{ij}b_{jk} = a_{i1}b_{1k} + a_{i2}b_{2k} + \cdots + a_{im}b_{mk} = a_{i*} \cdot b_{*k}.$$

The entry $c_{ik} = a_{i*} \cdot b_{*k}$ is the dot product of the ith row of A and the kth column of B:

$$\begin{bmatrix} \cdot & \cdot & \cdot \\ \boxed{1} & \boxed{4} & \boxed{2} \\ \cdot & \cdot & \cdot \end{bmatrix} \begin{bmatrix} \cdot & \boxed{3} & \cdot & \cdot \\ \cdot & \boxed{1} & \cdot & \cdot \\ \cdot & \boxed{1/2} & \cdot & \cdot \end{bmatrix} = \begin{bmatrix} \cdot & \cdot & \cdot & \cdot \\ \cdot & 8 & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \end{bmatrix}.$$

In the *row view of the matrix product* AB, 'the rows of A are coefficients in linear combinations of the rows of B':

$$\begin{bmatrix} 1 & 4 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 3 & 7 \\ 2 & 8 \end{bmatrix} = \begin{bmatrix} 1 \cdot \begin{bmatrix} 3 & 7 \end{bmatrix} + 4 \cdot \begin{bmatrix} 2 & 8 \end{bmatrix} \\ 2 \cdot \begin{bmatrix} 3 & 7 \end{bmatrix} + 3 \cdot \begin{bmatrix} 2 & 8 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 11 & 39 \\ 12 & 38 \end{bmatrix}.$$

In the *column view* we veiw the columns of B as providing coefficients in linear combinations of the columns of A.

$$\begin{bmatrix} 1 & 4 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 3 & 7 \\ 2 & 8 \end{bmatrix} = \begin{bmatrix} 3 & 7 \\ \times & \times \\ \begin{bmatrix} 1 \\ 2 \end{bmatrix} & \begin{bmatrix} 1 \\ 2 \end{bmatrix} \\ + & + \\ 2 & 8 \\ \times & \times \\ \begin{bmatrix} 4 \\ 3 \end{bmatrix} & \begin{bmatrix} 4 \\ 3 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 11 & 39 \\ 12 & 38 \end{bmatrix}$$

Practice

Compute the two products

$$\begin{bmatrix} a & b & c \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} \qquad \text{and} \qquad \begin{bmatrix} x \\ y \\ z \end{bmatrix} \begin{bmatrix} a & b & c \end{bmatrix}$$

and observe that **products do not commute**– the order of the matrices is important when we are multiplying them. These are two different products of two vectors. The first is called the *inner product* or the *dot product*, and the second is called the *outer product*.

Practice

Find the 3×3 matrix A such that

$$\begin{bmatrix} 3 & 2 & 0 \\ 1 & 2 & 2 \\ 4 & 2 & 1 \end{bmatrix} = A \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}.$$

Matrix times a vector

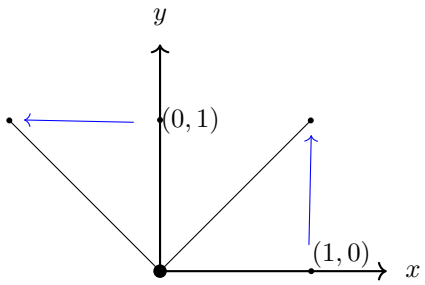
When the second matrix is a column vector $x \in \mathbb{C}^n$, multiplication

$$Ax = b$$

has two intepretations.

One interpretation is to view A as transforming the vector $x \in \mathbb{C}^n$ to a vector b in $\mathbb{C}^m$.

$$\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \qquad \text{and} \qquad \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$



It is a linear transformation:

$$A(x + y) = Ax + Ay \qquad \text{and} \qquad A(\alpha x) = \alpha Ax.$$

A good way to visualise what it is doing to the other points is to look at how it transforms the unit sphere S of all points that are distance 1 from the origin. In the above example we would transform the unit circle to the circle of radius $\sqrt{2}$.

Every linear transformation from $\mathbb{C}^n$ to $\mathbb{C}^m$ can be described as multiplication by an $m \times n$ complex matrix A.

The other interpretation views $b = Ax$ as the linear combination of columns of A with coefficients given by x:

$$\begin{bmatrix} 1 & 3 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = 2 \cdot \begin{bmatrix} 1 \\ 2 \end{bmatrix} + 1 \cdot \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 5 \\ 5 \end{bmatrix}.$$

Range and Nullspace of A

The *range*, range(A), of an $m \times n$ matrix A is the set of vectors in $\mathbb{C}^m$ that can be generated as linear combinations of its columns. It is clearly a subspace of $\mathbb{C}^m$ so more commonly called the *column space* of A. The *nullspace*, null(A), of A is the set of vectors x in $\mathbb{C}^n$ such that $Ax = 0$. This is also clearly a space, as $Ax = 0$ and $Ay = 0$ implies that

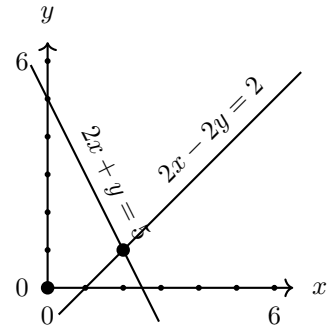
$$A(ax + by) = aAx + bAy = a0 + b0 = 0.$$

The range is the set of values b such that the equation $Ax = b$ has a solution and the nullspace is the set of values x such that $Ax = 0$ has a solution.

The equation $Ax = b$ and Gaussian Elimination

This is not in the text, but lets recall how we solve these equations in a traditional linear algebra course. A system of linear equations, each row of which can be viewed as a hyperplane in real space,

$$\begin{array}{rcrcl} 2x & - & 2y & = & 2 \\ 2x & + & y & = & 5. \end{array}$$



can be written as a matrix equation of the form $Ax = b$:

$$\begin{bmatrix} 2 & -2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 2 \\ 5 \end{bmatrix}.$$

In highschool you solved a system of equations like this. Your first step was to 'take a copy of the first equation from the second,' to get an equivalent system of equations.

$$\begin{array}{rcrcl} 2x & - & 2y & = & 2 \\ 2x & + & y & = & 5 \end{array} \quad \rightsquigarrow \quad \begin{array}{rcrcl} 2x & - & 2y & = & 2 \\ & & 3y & = & 3. \end{array}$$

The second system is the matrix equation

$$\begin{bmatrix} 2 & -2 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}.$$

You should understand how to solve matrix equations in this way, using 'row operations' to reduce this to something like

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

from which you can easily read of the solution $(x, y) = (2, 1)$. This is called Gaussian elimination. You might have seen it done using *augmented matrices* to reduce your work:

$$\left[\begin{array}{cc|c} 2 & -2 & 2 \\ 2 & 1 & 5 \end{array}\right] \rightsquigarrow \left[\begin{array}{cc|c} 2 & -2 & 2 \\ 0 & 3 & 3 \end{array}\right] \rightsquigarrow \left[\begin{array}{cc|c} 1 & -1 & 1 \\ 0 & 1 & 1 \end{array}\right] \rightsquigarrow \left[\begin{array}{cc|c} 1 & 0 & 2 \\ 0 & 1 & 1 \end{array}\right].$$

Sometimes things didn't work so nicely. Sometimes there was no solution, and sometimes there were many. We looked at rank and determinants and inverses to characterise when $Ax = b$ has a solution.

Rank

A set $V = \{v_1, \ldots, v_d\}$ of vectors in $\mathbb{C}^m$ is *(linearly) independent* if no one vector can be expressed as a linear combination of the others:

$$v_d = x_1 v_1 + x_2 v_2 + \cdots + x_{d-1} v_{d-1};$$

equivalently, if

$$(\begin{bmatrix} v_1 \mid v_2 \mid \cdots \mid v_d \end{bmatrix} x = 0) \to x = 0.$$

The *rank* of a matrix is the number independent columns it has. By this definition, the rank of an $m \times n$ matrix A is at most the minimum of m and n. If it is exactly $\min(m, n)$ it is *full rank*. A square $m \times m$ matrix A has full rank $\operatorname{rank}(A) = m$ if and only if the following equivalent conditions are true.

i. The columns of A are linearly independent.

ii. Gaussian elimination reduces A to the identity matrix I.

iii. A has an inverse matrix A^{-1} such that $AA^{-1} = I = \mathbb{A}^{-1}A$.

iv. The rows of A are linearly independent.

v. $\operatorname{range}(A) = \mathbb{C}^m$.

vi. $\operatorname{null}(A) = \{0\}$.

vii. $\det(A) \neq 0$.

viii. A is non-singular.

ix. 0 is not an eigenvalue of A.

Recall that the determinant $\det(A)$ is the product of the diagonals we get after doing Gaussian elimination on A without row switchs or scaling; we don't talk about it much in this class.

A square matrix is *singular* if it does not have full rank, and non-singular if it does; think of this as a definition.

A number λ is an *eigenvalue* of A if there is a non-zero solution to $Ax = \lambda x$. These we will talk about.

A matrix inverse times a vector

If we have the inverse A^{-1} of A, we can solve the equation $Ax = b$ by computing $x = A^{-1}b$. But we are usually not going to do this. Finding A^{-1} will often be computationally difficult. So the text urges you not to view $x = A^{-1}b$ as a transformation of b. They suggest that it is better viewed as a change of basis. When $A = \left[a_1|a_2|\dots a_n\right]$ has independent columns a_i, $x = (x_1, \dots, x_n)$ is the vector of co-efficients x_i such that

$$b = x_1a_1 + x_2a_2 + \cdots + x_na_n.$$

We view $x = A^{-1}b$ as these co-efficients of b expressed in a basis of range(A).

Problems from the text

Lecture 1: 1, 3, 4. (I can't understand # 2. Can anyone explain it?)

Extra for the exercises

When we were doing Gaussian elimination above we reduced

$$\left[\begin{array}{cc|c} 2 & -2 & 2 \\ 2 & 1 & 5 \end{array}\right] \qquad \text{to} \qquad \left[\begin{array}{cc|c} 2 & -2 & 2 \\ 0 & 3 & 3 \end{array}\right].$$

This was a row operation, we removed a copy of the first row from the second. We can view this as multiplying the first matrix, on the left, by the elementary matrix $E = \left[\begin{smallmatrix} 1 & 0 \\ -1 & 1 \end{smallmatrix}\right]$. Under the row view of matrix mutliplication, the first row of E says 'do nothing to the first row' and the second row of it says 'remove one copy of the first row from the second row':

$$\begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} \left[\begin{array}{cc|c} 2 & -2 & 2 \\ 2 & 1 & 5 \end{array}\right] = \left[\begin{array}{cc|c} 2 & -2 & 2 \\ 0 & 3 & 3 \end{array}\right].$$

Exercise 1 is asking you to start with a matrix B and do row and column operations. For the row operations, you will multiply by some matrix on the left, for the column operations, you will do so on the right. Be careful of the order.

Lecture 2 Orthogonal Vectors and Matrices

Sep. 11

The *transpose* of a matrix $A = [a_{ij}]$ is the matrix $A^T = [a_{ji}]$ that we get by reflecting it through its diagonal. A matrix A is *symmetric* if $A = A^T$.

Recall that a complex number is of the form

$$z = a + ib$$

where $a, b \in \mathbb{R}$ and i is the positive square root of -1. The *adjoint* a complex number $z = a + ib$ is $\bar{z} = a - ib$. More generally the *adjoint* of a matrix $A = [a_{ij}]$ is the matrix

$$A^* = [\bar{a}_{ji}]$$

that we get from it by taking its transpose, and replacing each entry with its adjoint. If $A^* = A$ then A is *hermitian* and a real hermitian matrix is *symmetric*.

Practice

For complex numbers a and b show that $\overline{ab} = \bar{a}\bar{b} = \bar{b}\bar{a}$. Using this show for complex matrices A and B that $(AB)^* = B^*A^*$.

Most (with some definite exceptions) of the theory of adjoints and hermitian matrices goes through without adjustment to transposes and symmetric matrics when we restrict to real matrices; so many books will use A^T much more than A^*, and often use A^H for A^*. Don't be surprised if I use T occasionally, instead of $*$.

Inner product

The *inner product* of two vectors $u = (u_1, \ldots, u_m)$ and $v = (v_1, \ldots, v_m)$ in $\mathbb{C}^m$ is the product

$$a^*b = (u_1v_1 + u_2v_2 + \cdots + u_mv_m) = \sum_{i=1}^{m} u_iv_i$$

where they are viewed as column vectors. This is also called the *dot product* of the vectors, so again, don't be surprised if I write it $a \cdot b$.

Practice

Compute u^*v when $u = (2, 3, 4)$ and $v = (3, 1, -1)$. When $u = (-i, 2, 1)$ and $v = (i, i, -2)$.

A vector $v = (v_1, \ldots, v_m)$ can be viewed as a point in $\mathbb{C}^m$, but also it can be viewed as the directed distance from 0 to that point. As such, its distance can be computed by the Pythagorean Theorem as

$$\|v\| = \sqrt{v^*v}.$$

A vector is *normal* if $\|v\| = 1$.

Practice

Compute $\|(3,4)\|$.

Using the Law of Cosines, one can show that where α is the angle between vectors a an b (viewed as arrows from the origin 0), that

$$\cos\alpha = \frac{a^*b}{\|a\|\|b\|}.$$

This is 0 for non-zero a and b if and only if the are perpendicular.

Orthogonal Vectors

Non-zero vectors a and b such that $a^*b = 0$ are called *orthogonal*. Sets $X, Y \subset \mathbb{C}^m$ of non-zero vectors *orthogonal* if $x^*y = 0$ for each $x \in X$ and $y \in Y$. A set Q of non-zero vectors is *orthogonal* if $x^*y = 0$ for any two vectors $x, y \in Q$; if the vectors are also normal, then Q is *orthonormal*.

When solving $Ax = b$ we are looking to express the vector b as a linear combination of the columns of A. This is especially easy when the columns of A are a set of orthonormal vectors A.

Indeed assume that $A = \begin{bmatrix} q_1 \mid \cdots \mid q_d \end{bmatrix}$ where the set $\{q_1, \ldots, q_d\}$ is orthonormal. If $Ax = b$ for $x = (x_1, \ldots, x_d)$, then we have that

$$b = x_1q_1 + \ldots x_dq_d$$

and we can find x_i by computing

$$q_i^*b = q_i^*(x_1q_1 + \ldots x_dq_d) = x_1q_i^*q_1 + \cdots + x_iq_i^*q_i + \cdots + x_dq_i^*q_d = x_i.$$

In particular, if there is a solution to $Ax = b$ then it is unique:

$$x = \sum_{i=1}^{d}(q_i^*b)q_i = \sum_{i=1}^{d}(q_iq_i^*)b.$$

The uniqueness gives us the following.

Fact 2.1. *A set of orthogonal vectors is independent.*

The two expressions $(q_i^*b)q_i$ and $(q_iq_i^*)b$ are the same but have different interpretations. In the first, q_i^*b is a scalar, it is the co-efficient of q_i in the expansion of b. In the second $(q_iq_i^*)$ is a matrix that projects b onto its ith component.

Even if there is no solution to $Ax = b$, we can write b as

$$b = x_1q_1 + \ldots x_dq_d + r$$

for some r that is orthogonal to the range of A.

Unitary Matrices

A square matrix U is *unitary* (or *orthogonal* in the real case) if its columns are orthanormal. In this case $U^*U = I$ and so $U^{-1} = U^*$. As a transformation, a unitary matrix is quite simple. It preserves the lengths of vectors and the angle between them, so is a rigid rotations of $\mathbb{C}^m$. Indeed

$$(Qu)^*(Qv) = u^*Q^*Qv = u^*Iv = u^*v$$

gives that $\|Qu\| = (Qu)^*(Qu) = u^*u = \|u\|$ and that the cosine of the angle between Qu and Qv is

$$\frac{(Qu)^*(Qv)}{\|Qv\|\|Qu\|} = (Qu)^*(Qv) = u^*v = \frac{u^*v}{\|u\|\|v\|}$$

which is the cosine of the angle between u and v.

Problems from the text

Lecture 2: 1-6. (1-4 are classical linear algebra theory.)

Lecture 3 Norms

In a classical linear algebra course we usually only consider one vector norm, the euclidean norm, because, in a real sense, all are equivalent. But numerically, other norms are often easier to work with. And when used to define matrix norms, something that is not always seen in a classical class, they have some interesting applications.

A norm is a non-negative function on vectors that has many properties similar to what you would expect of a 'length' function. Technically, a *(vector) norm* is a non-negative function $\|\cdot\| : \mathbb{C}^n \to \mathbb{R}^+$ that satisfies, for all $c \in \mathbb{C}$ and $v, w \in \mathbb{C}^n$,

- $\|v\| = 0$ if and only if $v = 0$, (positive definiteness),
- $\|c \cdot v\| = |c| \cdot \|v\|$, (scaling), and
- $\|v + w\| \leq \|v\| + \|w\|$, (the triangle inequality).

For any $p \geq 1$ the function

$$\|v\|_p = (\sum_{i=1}^{n} |v_i|^p)^{1/p}$$

is a norm. The most common cases we consider are when $p = 1$ or 2, or one defined by taking $p \to \infty$:

- the ℓ^1- or 1-*norm* $\|v\|_1 = |v_1| + |v_2| + \cdots + |v_n|$,
- the ℓ^2- or *Euclidean norm* $\|v\| = \|v\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$,
- the ℓ^∞- or *max norm* $\|v\|_\infty = \max\{|v_1|, |v_2|, \ldots, |v_n|\}$.

Problem:

Show that $\| \|_1, \| \|_2$ and $\| \|_\infty$ are norms.

Problem:

For each of the above norms $\| \|$, draw the unit sphere $S = \{v \colon \|v\| = 1\}$ in $\mathbb{R}^2$.

A common problem is to minimise or maximize a norm on the set of vectors in a subspace.

Problem:

Minimise the above norms on the vectors in the line $L : x/3 + y/4 = 1$.

When you do this, you will notice that the minimum point for the ℓ_1-norm is mostly 0s in such problems. While the other norms tend to have non-zero numbers in more coordinates. Because of this the ℓ_1-norm will often be a useful measure in approximations.

Apart from the p-norms, we also use weighted norms. For a (usually diagonal) non-singular matrix W let $\|v\|_W = \|Wv\|_2$.

Problem:

When $W = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$ draw the unit sphere S for the norm $\|\cdot\|_W$.

General matrix norms

Sep. 18

We also define norms $\|\ \|$ for matrices. We want that they have the same properties of positive definiteness, scaling, and triangle inequality that vector norms have, but we want that they also have sub-multiplicativity:

$$\|AB\| \leq \|A\|\|B\|.$$

Viewing an $m \times n$ matrix $A = [a_{ij}]$ as a vector of length mn:

$$(a_{11}, \ldots, a_{1n}, a_{21}, \ldots, a_{2n}, \ldots, a_{m1}, \ldots, a_{mn})$$

we immediatly get the first three properties, but we don't always get sub-multiplicativity. In the case of the 2-norm though, we do get multiplicativety by Cauchy-Schwarz, and so we do use this construction for the *Frobenius Norm.* Where $A = [a_{ij}]$ we define

$$\|A\|_F = \|(a_{11}, \ldots, a_{1n}, a_{21}, \ldots, a_{2n}, \ldots, a_{m1}, \ldots, a_{mn})\|_2.$$

Problem

Show that the Frobenius norm satisfies sub-multiplicativity.

Matrix norms induced by vector norms

While other vector norms don't give matrix norms in this way, they do give them in another way.

For any vector norm $\|\ \|$ let

$$\|A\| = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|} = \max_{x \in S} \|Ax\|$$

be the maximum stretch of x by A in this norm. Here S is the unit sphere with respect to the given norm.

Problem:

Show that submultiplicativity is satisfied above for any choice of the vector norm $\|\cdot\|$.

To compute the norm of a matrix A induced by a vector norm, we only have to look at all vectors in the unit sphere S under that vector norm, and check which one is stretched the most by A.

In the text they draw the action of a matrix A on the unit balls under the various vector norms, and show how one can visualise the induced matrix norms from these pictures.

As for the Frobinius norm, computing the 1-norm and the ∞-norm of a matrix are easy.

Example 3.1. To compute the 1-norm $\|A\|_1$ we for a matrix $A = [a_1 \mid \cdots \mid a_n]$ we maximise

$$\|Ax\| = x_1|a_1| + \cdots + x_n|a_n|$$

over x such that $x_1 + x_2 + \cdots + x_n = 1$. This is clearly maximised when x is the standard normal vector e_M with a 1 in the Mth coordinate where a_M is the largest column. So $\|A\|_1 = |Ae_M| = |a_M|$. The 1-norm of a matrix A is the maximum of the norms of its columns.

With similar analysis one can to the following.

Problem

Characterise the vectors in the unit sphere under the vector norm $\|\cdot\|_\infty$. Show that for a matrix A $\|A\|_\infty$ is the max, over all rows of A, of the sum of the entries in the row.

For other norms it is a bit harder computationally, though visually it is not so bad.

Example 3.2. Under the 2-norm, the unit sphere S is our usual unit sphere. Applying A to transfrom each point x in S to Ax transforms S into an paraboloid. The matrix norm $\|A\|_2$ is the length of the longest axis of this paraboloid.

Problems from the text

Lecture 3: 1-6

Background Material: Eigenvalues and Eigenvectors

An *eigenvalue* of a square matrix A is a constant λ such that

$$Ax = \lambda x$$

for some non-zero vector x. The vector x is the corresponding *eigenvector*.

Example 3.3. For the matrix $A = \left[\begin{smallmatrix} 1 & 0 \\ 0 & 3 \end{smallmatrix}\right]$, we have

$$A\left[\begin{smallmatrix} 1 \\ 0 \end{smallmatrix}\right] = \left[\begin{smallmatrix} 1 \\ 0 \end{smallmatrix}\right] \text{ and } A\left[\begin{smallmatrix} 0 \\ 1 \end{smallmatrix}\right] = \left[\begin{smallmatrix} 0 \\ 3 \end{smallmatrix}\right] = 3\left[\begin{smallmatrix} 0 \\ 1 \end{smallmatrix}\right],$$

so 1 is an eigenvalue with eigenvector $(1, 0)$ and 3 is another eigenvalue with eigenvector $(0, 1)$.

For if v_1 and v_2 are both eigenvectors with the eigenvalue λ then

$$A(av_1 + bv_2) = aAv_1 + bAv_2 = a\lambda v_1 + \lambda v_2 = \lambda(av_1 + bv_2)$$

and so the set of eigenvectors with a given eigenvalue λ are a space, called it *eigenspace*.

The matrix $A = \left[\begin{smallmatrix} 1 & 0 \\ 0 & 3 \end{smallmatrix}\right]$ has eigenspaces $\langle(1, 0)\rangle$ and $\langle(0, 1)\rangle$.

Practice

Show that A has no other eigenvectors. Indeed show that an $n \times n$ matrix can have at most n eigenvalues.

In this example it was easy to find the eigenvectors by inspection. Sometimes knowing what the matrix does as a transformation can also be used to find the eigenvectors and eigenvalues.

Practice

What are the eigenvectors of the rotation $A = \left[\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right]$?

In a classical linear algebra class we would find the eigenvalues and eigenvectors of a small matric algebraically by first finding the roots of what is called the 'characteristic polynomial' of the matrix. This is not practical for large matrices, so we will not do this. However there are some ideas we would get from this that are useful.

- A real matrix might have complex eigenvalues, and so complex eigenvectors.
- For a triangular matrix, the eigenvalues are the diagonal entries.

Basis Change and Similar Matrices and Diagonalisation

An $m \times m$ matrix $D = [d_{ij}]$ whose only non-zero enties are the diagonal entries d_{ii} is a *diagonal matrix*. These are quite nice to work with. As transformations they simply stretch the standard normal vector e_i by a factor of d_{ii}. When we transform the unit sphere $S = \{v \in \mathbb{C}^m \mid \|v\| = 1\}$ by D we get a hyperellipse DS whose i-axis has length exactly d_{ii}.

A more general $m \times m$ matrix A might not transform S as nicely. It generally has some rotating and some stretching. A change of basis can often transform all vectors so that the action of A is nicer. To understand the action of A we transfoem the vectors, do the nice action, and then transform them back. Algebraically this looks like writing A as $A = BXB^{-1}$ for an invertible matrix B. The action Ax of A on a vector x decomposes into $BXB^{-1}x$ where B^{-1} transforms the vector, X does the nice action and then B transforms us back to the original setting.

Note

I find change of basis one of the most difficult to explain 'easy' idea. It is personal, if you don't understand my explanation, try the one in the book. Try other books.

A diagonalisation of A is the nicest example of a change of basis transformation. If $Ax = \lambda x$ and B is invertible, then for the vector Bx we have

$$BAB^{-1}(Bx) = BAx = B\lambda x = \lambda Bx.$$

So A and BAB^{-1} have the same eigenvalue λ. (Its eigenvector under A is x, and its eigenvector under BAB^{-1} is Bx.) Two $m \times m$ matrices are *similar* if they have the same eigenvalues.

If A has a full set of eigenvectors, that is, m independent eigenvectors $\{v_1, \ldots, v_m\}$, then we have

$$A[v_1|v_2|\ldots|v_m] = [\lambda_1 v_1|\ldots|\lambda_m v_m] = [v_1|v_2|\ldots|v_m]\begin{bmatrix}\lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_m\end{bmatrix}.$$

Letting X be the matrix whose i^{th} column is the ith eigenvector v_i and Λ be the diagonal matrix whose i^{th} entry is λ_i, we get the *diagonalisation*

$$A = X\Lambda X^{-1}$$

of A.

Problem

Show that where A has diagonalisation $A = X\Lambda X^{-1}$, and S is the unit sphere, AS is the hyperellipse that has radius λ_i in the direction of v_i, for all i.

The diagonalisation has some nice applications. We can easily compute powers A^n of A:

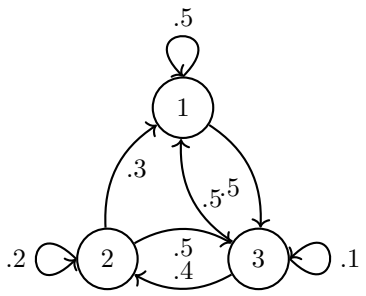
$$A^p = (X\Lambda X^{-1})^p = X\Lambda^p X^{-1} = X\begin{bmatrix}\lambda_1^p & & & \\ & \lambda_2^p & & \\ & & \ddots & \\ & & & \lambda_n^p\end{bmatrix}X^{-1}.$$

We use this in the following example.

Example 3.4. The matrix

$$T = \begin{bmatrix}.5 & .3 & .5 \\ 0 & .2 & .4 \\ .5 & .5 & .1\end{bmatrix}$$

is the transtision matrix for the Markov process described by the following state diagram:



There are three states: $1, 2$ and 3. A *state* vector $u(t) = (p_1(t), p_2(t), p_3(t))$ lists the probability $p_i(t)$ that we are in state i at time t. The state digram, and the transition matrix T, show the probablility t_{ij} that, if in state i at time t, we will be in state j at time $t + 1$.

So

$$u(t+1) = Tu(t),$$

and for some initial state $u(0)$ we have that

$$u(t) = T^t u(0),$$

To 'solve' the Markov Process, we want to find the limiting state vector $u(t)$ as t goes to infinity.

Well we can find the eigens of T:

$$\begin{aligned} \lambda_1 &= -2/3 & v_1 &= (1, 2, -3) \\ \lambda_2 &= 1/5 & v_2 &= (1, -1, 0) \\ \lambda_3 &= 1 & v_3 &= (13, 5, 10) \end{aligned}$$

We notice that if $p(0) = \frac{1}{28}(13, 5, 10)$ our initial state has eigenvalue one, and so we stay in that state for ever. But what happens if we start in some other state?

To see, we first diagonalise T:

$$T = S\Lambda S^{-1} = \frac{1}{84}\begin{bmatrix} 1 & 1 & 13 \\ 2 & -2 & 5 \\ -3 & 0 & 10 \end{bmatrix}\begin{bmatrix} -2/3 & & \\ & 1/5 & \\ & & 1 \end{bmatrix}\begin{bmatrix} 10 & 10 & -18 \\ 35 & -49 & -21 \\ 3 & 3 & 3 \end{bmatrix}.$$

Then we can compute $u(t) = T^t u(0) = X\Lambda^t X^{-1} u(0)$:

$$\begin{aligned} u(t) &= \begin{bmatrix} 1 & 1 & 13 \\ 2 & -2 & 5 \\ -3 & 0 & 10 \end{bmatrix}\begin{bmatrix} -2/3 & & \\ & 1/5 & \\ & & 1 \end{bmatrix}\begin{bmatrix} 5/42 & 5/42 & -3/14 \\ 5/12 & -7/12 & -1/4 \\ 1/28 & 1/28 & 1/28 \end{bmatrix}\begin{bmatrix} p_1 \\ p_2 \\ p_3 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 1 & 13 \\ 2 & -2 & 5 \\ -3 & 0 & 10 \end{bmatrix}\begin{bmatrix} -2/3 & & \\ & 1/5 & \\ & & 1 \end{bmatrix}\begin{bmatrix} 5p_1/42+5p_2/42-3p_3/14=:c_1 \\ 5p_1/12-7p_2/12-1p_3/4=:c_2 \\ 1p_1/28+1p_2/28+1p_3/28=:c_3 \end{bmatrix} \\ &= c_1(-2/3)^t\begin{bmatrix} 1 \\ 2 \\ -3 \end{bmatrix} + c_2(1/5)^t\begin{bmatrix} 1 \\ -2 \\ 0 \end{bmatrix} + c_3(1)^t\begin{bmatrix} 13 \\ 5 \\ 10 \end{bmatrix} \end{aligned}$$

Now as t gets big, the first two disappear, and so

$$u_t \to c_3 \begin{bmatrix} 13 \\ 5 \\ 10 \end{bmatrix} = \frac{1}{28} \begin{bmatrix} 13 \\ 5 \\ 10 \end{bmatrix},$$

which is our steady state. We don't even have to compute c_3, as we know that this must be a state vector, so we can scale it to make it one. The initial position $u(0)$ didn't even matter. The other two terms go to 0, and so c_3 is non-zero and we will definitely converge to the state go to the state $\frac{1}{28}(13, 5, 10)$ that we got by normalising the largest eigenvector.

Lecture 4 The Singular Value Decomposition

In a classical linear algebra class we concentrate on the eigenvalues λ_i and eigenvectors v_i of a square matrix A– the objects such that

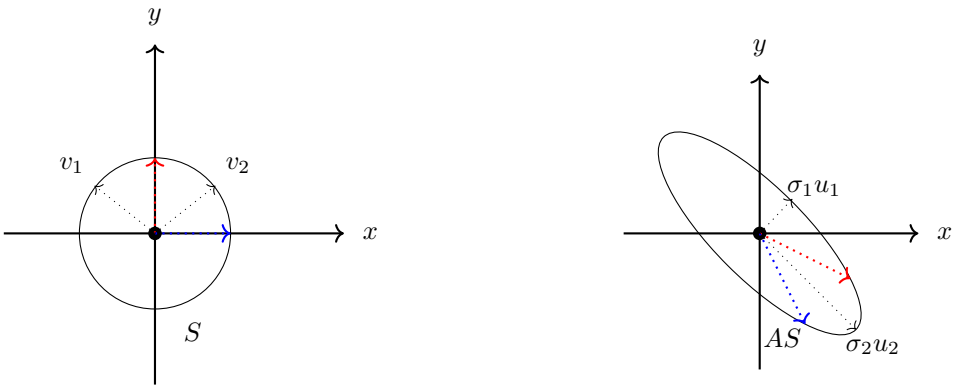
$$Av_i = \lambda_i v_i.$$

We use these to (hopefully) diagonalise the matrix. When the matrix is symmetric these eigenvalues would all be non-negative reals. When they are all positive, then the matrix has a full set of orthogonal eigenvectors, and we have a diagonalisation of the matrix by an orthogonal matrix. We have a lot of nice applications in this case, and the action of the matrix as a transformation is transparent. It maps the unit sphere to a hyperellipse, the eigenvector of the largest eigenvalue becoming the main axis.

This is a quick recall of a lot of theory, but we are not going to cover this. We aren't concentrating on square matrices, our matrices often don't have eigenvalues; and even when they are square, we might not have a full set of eigenvalues.

Singular values are a bit more forgiving, the always exist, even for non-square matrices, and their corresponding singular-vectors can always be taken to be orthogonal. We present them by starting from the notion of the hyperellipse.

Let A be an $m \times n$ matrix. As a transformation it takes vectors in $\mathbb{C}^n$ to $\mathbb{C}^m$. For this discussion, it is probably easier to think of it as a real matrix, taking vectors in $\mathbb{R}^n$ to $\mathbb{R}^m$. As it is linear, we can interpolate its action on the unit sphere S in $\mathbb{R}^n$ by computing its action on the standard normal vectors. This is perhaps more intuitive when A has full rank and $m > n$– it is not to hard to see that the action of A takes S in $\mathbb{R}^n$ to some hyperellipse AS in $\mathbb{R}^n \subset \mathbb{R}^m$.



To describe this hyperellipse AS, we need its axes $\sigma_i u_i$, and to describe the transformation we also need to know the vectors v_i of S such that $Av_i = \sigma_i u_i$.

The values σ_i, which are usually ordered in non-increasing order

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_n,$$

are the *singular values* of A. The vectors $u_1, \ldots, u_n$ are the *left singular vectors*, and the vectors $v_1, \ldots, v_n$ are the *right singular vectors*. As $Av_i = \sigma u_i$ for each i we have that

$$A\begin{bmatrix} v_1 \mid \cdots \mid v_n \end{bmatrix} = \begin{bmatrix} u_1 \mid \cdots \mid u_n \end{bmatrix} \begin{bmatrix} \sigma_1 & & & \\ & \sigma_2 & & \\ & & \ddots & \\ & & & \sigma_n \end{bmatrix},$$

and so writing these matrices as $V, \hat{U}$ and $\hat{\Sigma}$, where V and $\hat{U}$ will both turn out to have orthogonal colums, so, in particular $V^{-1} = V^*$, we get that

$$A = \hat{U}\hat{\Sigma}V^*.$$

This is called the *reduced singular value decomposion (reduced SVD)* of A. The hats on $\hat{U}$ and $\hat{\Sigma}$ are because though this is the one we use more, there is a 'full SVD' which we write without hats. The order of the matrices in the decomposition explain the use of 'left' and 'right' for the names of the special vectors.

Indeed, specifying the dimensions of the matrices, we have

$$A_{m\times n} = \hat{U}_{m\times n}\hat{\Sigma}_{n\times n}V^*_{n\times n}.$$

Padding the bottom of $\hat{\Sigma}$ with rows of 0 so that it is $\Sigma_{m\times n}$ we can extend $\hat{U}$ to U by adding $m-n$ more orthogonal columns, and so make U square, and so unitary (orthogonal in the real case), too. Doing so we get the full SVD decomposition

$$A = U\Sigma V^* = U_{m\times m}\Sigma_{m\times n}V^*_{n\times n}$$

of A– because the extra rows in Σ are all 0, the added columns of U do not change the product.

If $m < n$ then we pad $\hat{\Sigma}$ with extra columns of 0, and V^* with extra rows.

Formal Definition

As for square matrices, we call an $m \times n$ matrix $A = [a_{ij}]$ diagonal if the only non-zero entries are the a_{ii}.

A full SVD of an $m \times n$ matrix A is a factorisation $A = U\Sigma V^*$ such that U and V^* are unitary, and Σ is diagonal with non-negative real numbers $\sigma_1 \geq \cdots \geq \sigma_{\min(m,n)}$ on the diagonal.

The following is proved in the text.

Theorem 4.1. *Every complex matrix A has a full SVD. Further, the matrix Σ is unique, and if it is square and the σ_i are distinct, then the columns of U and V and uniquely determined upto multiplication by scalar factors ± 1.*

This σ_1 was the stretch of our main axis, the amount that A stretches v_1. This is the most any unit vector is stretched, in the 2-norm, so

$$\|A\|_2 = \sigma_1.$$

Problems from the text

Lecture 4: 1-5 (For question 3, use any programming language.)

Lecture 5 More on the SVD

As for a diagonalisation $A = X\Lambda X^{-1}$ of a square matrix, the SVD $A = U\Sigma V^*$ of a general matrix A can be viewed as a change of basis. But after changing to the nice basis, doing the diagonal action, we don't change back to the original basis, we finish off by changing to some orthogonal basis of the co-domain of A as a transformation.

We lose the fact that we end up in the standard normal basis in the codomain as would naturally occur by viewing A as a transformation, but we still end up in a orthonormal basis defined by U, *and* this works for *any* matrix.

Matrix Properties via the SVD

Let A have SVD

$$A = U\Sigma V^*.$$

Recalling that the rank of a matrix is the number independent columns, or the number of independent rows that it has, we get the following.

Theorem 5.1. *The rank r of a matrix A is the number of non-zero singular values it has.*

Proof. We know that the rank of a product of matrices is the minimum of the ranks of the matrices. In the SVD $A = U\Sigma V^*$ the matrices U and V^* both have full rank, m and n respectively, and the rank of sigma Σ is at most the min of these, and being diagonal, is equal to the number of non-zero entries, which is the number of non-zero singular values. □

So the rank r is some number not greater than $\min(m, n)$. The SVD makes explicit the range and nullspace of A.

Theorem 5.2. *Where $A = U\Sigma V^*$ and u_i is the ith column of U, the set $\{u_1, \ldots, u_r\}$ is a basis of the range of A, and the set $\{u_{r+1}, \ldots u_m\}$ is a basis of its nullspace.*

Proof. As the rows below the rth of Σ are 0 the columns of U can change to 0 without changing A, so are not it its range. By rank, the other columns span its range, and being orthogonal, they are independent, so a basis of it. The remaining columns of U, being orthogonal to the range of A, are in its nullpace, and being orthogonal, are a basis for it. □

We saw above that $\|A\|_2 = \sigma_1$. The following observation is also immediate from the fact that the Frobenius norm is the 2-norm of A as a vector, and vectors do not stretch when multiplied by a unary matrix.

Fact 5.3. *For any matrix $A = U\Sigma V^*$ we have $\|A\|_F = \|\Sigma\|_F = (\sum_{i=1}^r \sigma_i^2)^{1/2}$.*

Now, the matrixes AA^* and A^*A are square, so have eigenvalues and eigenvectors. When $A = U\Sigma V^*$ we get

$$A^*A = V\Sigma^*U^*U\Sigma V^* = V\Sigma^*\Sigma V^*$$

where the $\Sigma^*\Sigma$ is diagonal with entries σ_i^2. This is a diagonalisation by orthogonal V, and so we know that $(A^*A)v_i = \sigma_i^2 v_i$ showing that $\lambda_i = \sigma_i^2$ is an eigenvalue of A^*A with eigenvector v_i.

A similar argument works for AA^* and we get the following.

Fact 5.4. *Let A have SVD $A = U\Sigma V^*$.*

- *The singular values σ_i of A are the positive square roots of the non-zero eigenvalue λ_i of A^*A and AA^*.*
- *The right singular vectors $v_1, \dots v_r$ of A are the eigenvectors of A^*A.*
- *The left singular vectors $u_1, \dots, u_r$ of A are the eigenvectors of AA^*.*

When A is hermitian, so $A^* = A$, (symmetric if A is real), then $A^* = (U\Sigma V^*)^* = V\Sigma^*U^*$ and

$$A^2 = AA^* = U\Sigma V^*V\Sigma^*U^* = U|\Sigma|^2U^* = U|\Sigma|U^*U|\Sigma|U^*$$

and so

$$A = U|\Sigma|U^*$$

where $|\Sigma|$ is just Σ where we have taken absolute values of the entries.

Fact 5.5. *If A is Hermitian, (real symmetic), then its singular values are the absolute values of its eigenvalues.*

Low rank approximation

Oct. 02

One useful notion that is not always seen in a conventional linear algebra course is that of a low rank approximation of a matirx.

Another view of the matrix product $A_{\ell\times m}B_{m\times n} = [c_{ik}]$ is the outer product view. We saw that $c_{ik} = a_{i*}^T b_{*k}$. Fixing some j, take the jth term $a_{ij}b_{jk}$ of $c_{ik} = a_{i*}^T b_{*k}$. Put this, for all i and k in the ith row and k^{th} column of a matrix C_j– so this is an $\ell \times n$ matrix with ik^{th} entry $a_{ij}b_{jk}$. This matrix C_j is exactly the outer product $a_{*j}b_{j*}^T$.

Thus we can write

$$AB = \sum_{j=1}^m a_{*j}b_{j*}^T = \sum_{i=1}^m C_j$$

as a sum of outer product matrices.

The matrix C_j has rank one– all its columns are scales of a_{*j}. When we are working with big matrices, often an approximation will do, and low rank ones can often take up much less space, and still contain much of the important information in the matrix. We just want to pick the right C_j.

The SVD allows us to do this in a nice way. Let A have SVD $A = U\Sigma V^*$. As above we get the outer product sum

$$A = \sum_{i=1}^{r} \sigma_i u_i v_i^*.$$

In our SVD, the entries of Σ are in non-decreasing order, and the columns u_j and v_j are all normal, so the norms of the factors $\sigma_i u_i v_i^*$ are also non-decreasing. Cutting them off at some $\nu < r$ the approximation

$$A_\nu = \sum_{i=1}^{\nu} \sigma_i u_i v_i^*$$

is $A_\nu = U\Sigma_\nu V^*$ where Σ_ν is the matrix we get from Σ by replacing the first ν singular values with 0. So it has rank ν, and is a good rank ν approximation of A. In fact, the following theorem of Eckart and Young says, it is the best rank ν approximation of A.

Theorem 5.6. *Let A be a rank r matrix, and $\nu \leq r$. For any rank ν matrix B of the same dimensions we have*

$$\sigma_{\nu+1} = \|A - A_\nu\|_2 \leq \|A - B\|_2.$$

Proof. As $A - A_\nu$ consists of the last $r - \nu$ factors of $A = \sum_{i=\nu+1}^{r} \sigma_i u_i v_i^*$, its largest special value is $\sigma_{\nu+1}$ and so $\|A - A_\nu\| = \sigma_{\nu+1}$.

Now where $A = A_{m\times n}$ let B be an $m \times n$ matrix of rank at most ν. The null-space of B has dimension at least $n - \nu$, so intersects the column space of $A_{\nu+1}$, which has dimension $\nu + 1$, non-trivially. Let u be some unit vector in the intersection. So $(A - B)u = Au = A_{\nu+1}u$, and so

$$\|A - B\|_2 \geq \|(A - B)u\|_2 = \|A_{\nu+1}u\| \geq \sigma_{\nu+1}$$

□

We don't yet know how to compute the SVD of a matrix. We will get there, but first we look as some other ideas. These are things we can compute with the SVD, but they are also ideas that can help us in computing it.

You might have noticed that we can compute the SVD by finding the eigenvalues and eigenvectors of A^*A. This is an important idea, but even if you remember how to find the eigenvectors of a square matrix algebraically, this not usually how we will do it. When A is wide, A^*A is much bigger than A, so working with it will take more computation, this is something we are going to want to avoid.

Problems from the text

Lecture 5: 3,4
For #3, we haven't really discussed finding singular values and singular vectors, but have mentioned you can use eignvalues and eigenvectors of A^*A and AA^*. If you don't know how to do this, look at the extra material between Lecture 3 and 4. Try this and check it with a computer, or use a computer to find the eigenvalues. Then try to do the rest of the question by hand.

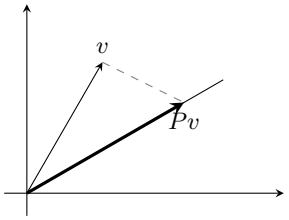
Question #4 take less knowledge, but some playing around. It is asking you to find eigenvalues λ_i and eigenvectors x_i of $B = \begin{bmatrix} 0 & A^* \\ A & 0 \end{bmatrix}$: $Bx_i = \lambda_i x_i$. Using that $Av_i = \sigma_i u_i$ and that $A^*u_i = \sigma_i v_i$ you should be able to find that each singular value σ_i is an eigenvalue of B. This gives you m eigenvalues/vectors. To get the other m, notice that you also have that $A(-v_i) = -\sigma_i u_i$; you should be able to show that $-\sigma_i$ is also an eigenvalue of B.

Part II: QR-factorisation and Least Squares

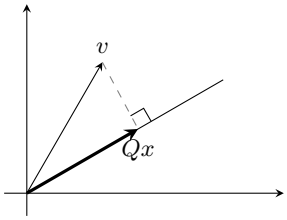
Lecture 6 Projectors

A square matrix P is *idempotent* or a *projector* if $P^2 = P$. Matrices P whose action as a linear transformation projects $\mathbb{C}^m$ onto a subspace are projectors– once we have projected onto the subspace, applying the action again does nothing.

A projector P projects a vector v to the vector Pv in its column space range(P). The direction it projects v is $Pv - v = (P - I)v$. The projection can be *oblique* or *orthogonal*.



Oblique projection



Orthogonal projection

Fact 6.1. *For any projector P, we have* range$(P - I) =$ null(P).

Proof. Let v be in range$(P - I)$, so $v = (P - I)w$ for some w. Then

$$Pv = P(P - I)w = (P^2 - P)w = 0,$$

sheing that v is in null(P). Let $v \in$ null(P). Then $(P - I)v = Pv - v = -v$, showing $v \in$ *range*$(P - I)$. □

Problem 6.1:

Show that if P is a projector, then so is $I - P$

Problem

Show for an $m \times m$ projector P that $A =$ range P and $B =$ range$(I - P)$

i. generate $\mathbb{C}^m$: any $v \in \mathbb{C}^m$ can be written $v = a + b$ for $a \in A$ and $b \in B$, and

ii. have trivial intersection: $v \in$ range $A \cap$ range B implies $v = 0$.

That they satisfy these conditions means they are *complementary*.

Orthogonal Projectors

Mostly we are interested in projectors P for which $Pv - v$ is orthogonal to Pv. A projection P is *orthogonal* if range$(P - I)$ is orthogonal to range(P); equivalently, if $P^*(P - I) = 0$.

Algebraically, these are the projectors that are Hermitian.

Fact 6.2. *A projector P is orthogonal if and only if $P^* = P$.*

Proof. If $P^* = P$. Then $P^*(P - I) = P(P - I) = P^2 - P = P - P = 0$, so P is orthogonal.

On the other hand assume that P is orthogonal. Take an orthanormal basis $\{q_1, \ldots, q_r\}$ of range(P) and an orthanormal basis $\{q_{r+1}, \ldots, q_m\}$ of range$(P - I) = \text{null}(P)$. Because P is orthogonal, these bases are orthogonal, so make an orthanormal basis of $\mathbb{C}^m$. Letting Q be the unitary matrix $Q = \left[q_1 | \ldots | q_m\right]$, We get

$$PQ = \left[q_1 | \ldots | q_r | 0 | \ldots | 0\right].$$

So $\Sigma = Q^*PQ$ is diagonal with 1 in the first r spots and 0 elsewhere. This gives us an SVD $P = Q\Sigma Q^*$, which is Hermitian.

□

Problem

Show that the complement $I - P$ of an orthogonal projector P is also orthogonal.

6.1 Projection with an Orthonormal Basis

For an orthogonal projection P, we saw in the proof above that the SVD gives $P = Q\Sigma Q^*$. Using the reduced SVD, Σ becomes the identity, $I_{r\times r}$, and we can write our orthogonal projector

$$P_{\hat{Q}} := \hat{Q}\hat{Q}^*.$$

The hat should remind you that $\hat{Q}$ isn't square. If it were then this is all just I, but this is only the identity for vectors in the range of $\hat{Q}$. Indeed, other vectors are killed, so it is the projector onto range$(\hat{Q})$.

A useful case of this is when we take $\hat{Q}$ as a single unit vector q. The matrix $P_q = qq^*$ projects vectors onto q.

To project onto a non-unit vector a, we can first normalise it:

$$P_a = \frac{aa^*}{a^*a}.$$

The projection of v onto a is thus $P_a v = \frac{aa^*}{a^*a}v$. Computationally it is often easier not to compute P_a but to compute

$$\text{proj}_a(v) := \frac{aa^*}{a^*a}v = \frac{(a^*v)}{a^*a}a.$$

Problem

Compute the projection of the vector $v = (3, 4)$ onto the vector $a = (1, 1)$.

6.2 Projection with an arbitrary Basis

Just like $P_{\hat{Q}} = \hat{Q}\hat{Q}^*$ projects orthogonally onto range$(\hat{Q})$, we can find, for any matrix A, a matrix P_A that orthogonally projects onto range(A).

We may assume that A has independent columns. To find a matrix P such that Pv is the orthogonal projection of v onto range(A), we need that $Pv - v$ is orthogonal to each column of A:

$$A^*(Pv - v) = 0.$$

Now Pv is in range(A), so can be written as Ax for some vector x. Thus we have $A^*(Ax - v) = 0$ giving that $A^*Ax = A^*v$. As A has full rank, A^*A is invertible, and so we get $x = (A^*A)A^*v$ and so

$$Pv = Ax = A(A^*A)A^*v.$$

The matrix $P_A := A(A^*A)A^*$ is the *projection matrix* for A; it projects any vector v orthogonally onto range(A).

Problems from the text

Lecture 6: 1-5

Lecture 7 QR Factorisation

7.1 Gram Schmidt Orthogonalisation

Given a basis $\{a_1, \ldots, a_n\}$ of a subspace $\langle a_1, \ldots, a_n \rangle$ of $\mathbb{C}^m$, the Gram-Schmidt process is the basic algorithm for finding an equivalent orthonormal basis $\{q_1, \ldots, q_n\}$. Equivalent means that

$$\langle q_1, \ldots, q_n \rangle = \langle a_1, \ldots, a_n \rangle$$

but Gram-Schmidt gives us more: it gives us $\{q_1, \ldots, q_n\}$ so that

$$\langle q_1, \ldots, q_i \rangle = \langle a_1, \ldots, a_i \rangle$$

for each $i = 1, \ldots, n$.

Example 7.1. Let $V \subseteq \mathbb{R}^3$ be generated by the basis

$$\{a_1 = (1, 0, 1), a_2 = (1, 0, 0), a_3 = (2, 1, 0)\}.$$

We use the Gram-Schmidt process to get another orthanormal basis $q_1, \ldots, q_3$

- $q_1 = a_1/\|a_1\| = \frac{1}{\sqrt{2}}(1, 0, 1)$
- $q_2' = a_2 - (q_1^* a_2)q_1 = (1, 0, 0) - \frac{1}{2}(1, 0, 1) = \frac{1}{2}(1, 0, -1)$
- $q_2 = \frac{1}{\sqrt{2}}(1, 0, -1)$
- $q_3' = a_3 - (q_1^* a_3)q_1 - (q_2^* a_3)q_2 = \cdots = (0, 1, 0)$
- $q_3 = (0, 1, 0)$.

Where A is the matrix $A = \begin{bmatrix} a_1 \mid a_2 \mid a_3 \end{bmatrix}$ with these basis vectors as columns, we

$$A = \begin{bmatrix} 1 & 1 & 2 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} q_1 \mid q_2 \mid q_3 \end{bmatrix} \begin{bmatrix} \sqrt{2} & \frac{1}{2} & q_1^* a_3 \\ 0 & \frac{1}{\sqrt{2}} & q_2^* a_3 \\ 0 & 0 & q_3^* a_3 \end{bmatrix}$$

Algorithm 7.2. *(Gram Schmidt)*

Input: Independent vectors $\{a_1, \ldots, a_n\}$.

Output: Pairwise orthogonal unit vectors $\{q_1, \ldots, q_n\}$.

Algorithm: For $i = 1, \ldots, n$ compute:

i. $q_i' = a_i - \sum_{j=1}^{i-1} \text{proj}_{q_j} a_i = a_i - \sum_{j=1}^{i-1} (q_j^* a_i) q_j$

ii. $q_i = q_i/\|q_i\|$

Note

Projecting a vector onto q_i' is the same as projecting onto q_i, so for Gram Schmidt, we can do the whole thing without normalising as we go. Sometimes it is useful to only normalise the vectors q_i' at the end. But be careful if you do this, the projection formula doesn't simplify, you need to keep the denominator: $\text{proj}_{q_i'} a_j = \frac{(q_i')^* a_j}{(q_i')^*(q_i')} q_i'$

In general, we saw that an orthogonal basis $\{q_1, \ldots, q_n\}$ allows us write a vector in the space as a linear combination of basis vector:

$$v = (q_1^* v)q_1 + (q_2^* v)q_2 + \cdots + (q_n^* v)q_n$$

As we got these basis vectors from $a_1, \ldots, a_n$ via Gram Schmidt, we get

$$a_i = (q_1^* v)q_1 + \cdots + (q_i^* v)q_i + 0 + 0 + \cdots + 0.$$

Oct. 16

This gives us the $\hat{Q}\hat{R}$-decomposition of A

$$\underbrace{\begin{bmatrix} a_1 \mid a_2 \mid \cdots \mid a_n \end{bmatrix}}_{A} = \underbrace{\begin{bmatrix} q_1 \mid q_2 \mid \cdots \mid q_n \end{bmatrix}}_{Q} \underbrace{\begin{bmatrix} q_1^* a_1 & q_1^* a_2 & \ldots & q_1^* a_n \\ 0 & q_2^* a_2 & \ldots & q_2^* a_n \\ & & \ddots & \\ 0 & 0 & \ldots & q_n^* a_n \end{bmatrix}}_{R}$$

of A where R is a real upper triangular matrix.

This again is a *reduced* QR-decomposition of A– the matrix $\hat{Q}$ is not square. Again we can add orthogonal columns to $\hat{Q}$ to make it orthonormal, and rows of 0 to $\hat{R}$ to compensate. This gives us the full QR-decomposition $A = QR$ of A into an orthonormal Q and upper-triangular R.

If A does not have independent columns, we still get the reduced QR decomposition. Where we have a column a_i that is a linear combination of the previous columns, the corresponding q_i should be taken as 0 in the Gram-Schmidt process, but then at the end, replaced with any unit vector normal to the 'real' columns of Q. The ith row of R will be a 0 row.

7.2 When vectors become continuous

We can view a column vector v as a function $\mathbb{Z}^+ \to \mathbb{C}$ taking i to its ith entry. In the same way, we can view a function $f : \mathbb{R} \to |C$ as a infinitely long column vector. A common setup for this is to use functions in $L^2[-1, 1]$, the set of functions $f : [-1, 1] \to \mathbb{C}$, with the inner product

$$u^* v := \int_{-1}^{1} \bar{u} v \, \mathrm{d}x.$$

This defines a norm $\|v\| = \sqrt{v^* v}$ and so a projection formula $\text{proj}_v(u) = \frac{v*u}{v^* v} v$ for projecting u onto v.

For a matrix $A = [a_0 \mid \cdots \mid a_n]$ where each a_i is a function in $L^2[-1,1]$ we can then apply Gram Schmidt to get a QR decomposition

$$A = \begin{bmatrix} q_0 \mid q_1 \mid \cdots \mid q_n \end{bmatrix} \begin{bmatrix} r_{00} & r_{01} & \dots & r_{0n} \\ 0 & r_{11} & \dots & r_{1n} \\ & & \ddots & \\ 0 & 0 & \dots & r_{nn} \end{bmatrix}.$$

Example 7.3. Let $A = \begin{bmatrix} 1 \mid x \mid x^2 \end{bmatrix}$ be a 'matrix of infinite columns'. To apply Gram Schmidt we normalise $a_1 = 1$ to $q_1 = 1/\sqrt{2}$:

$$\|a_1\|^2 = \int_{-1}^{1} 1 \cdot 1 \, \mathrm{d}x = 1 - (-1) = 2.$$

It is nicer to work with $q_1 = 1$ than $q_1 = 1/\sqrt{2}$– this is one of those cases where we will do Gram Scmidt without normalising. We take $q_1 = 1$.

Projecting $a_2 = x$ onto $q_1 = 1$ gives

$$\mathrm{proj}_{q_1}(a_2) = \frac{(q_1^* a_2)}{q_1^* q_1} q_1 = \int_{-1}^{1} \frac{1}{\sqrt{2}} x \, \mathrm{d}x = 0.$$

They are already orthogonal, so $q_2 = a_2 = x$.

Projecting $a_3 = x^2$ onto $q_1 = 1$ and $q_2 = x$ we get

$$\mathrm{proj}_{q_1}(x^2) = (\int_{-1}^{1} x^2 \, \mathrm{d}x)/q_1^* q_1 = (2/3)/2 = 1/3$$

$$\mathrm{proj}_{q_2}(x^2) = (\int_{-1}^{1} x^3 \, \mathrm{d}x)/q_2^* q_2 = 0.$$

So $q_3 = x^2 - \frac{1}{3}$.

We end up with a QR-decompositon like of A

$$A = \begin{bmatrix} 1 \mid x \mid x^2 \end{bmatrix} = \begin{bmatrix} 1 \mid x \mid x^2 - \frac{1}{3} \end{bmatrix} \begin{bmatrix} 1 & 0 & \frac{1}{3} \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

It isn't quite a QR-decomposition because the columns of Q are not normal, but they are orthogonal.

Taking $a_i = x^i$ for higher i, these q_i, scaled so that $q_i(1) = 1$ are known as the Legendre polynomials P_i:

$$P_0(x) = 1 \qquad P_1(x) = x \qquad P_2(x) = \frac{2}{3}x - \frac{1}{3} \qquad P_3(x) = \frac{5}{2}x^3 - \frac{3}{2}x \qquad \dots.$$

One often sees a function in $L^2[-1,1]$ projected onto them to express it as a linear combination of them. Apparently this is useful in solving PDEs.

Solving $Ax = b$ via a QR-decomposition

One of the main goals in linear algebra is solving an equation $Ax = b$. One of our goals is doing so without using Gaussian Elimination, and without inverting A.

When $A = QR$ then this becomes $QRx = b$ which is equvalent to $Rx = Q^*b$; recall that $Q^* = Q^{-1}$ because Q is orthogonal. So it is enough to compute $y = Q^*b$ and to solve

$$Rx = y.$$

This is easier than solving $Ax = b$ because R is upper triangular– Gaussian Elimination is already done for us, we only have to do the easier 'clearing above the diagonal' part. We will talk soon about why this is easier. We also need to discuss why Gram Schmidt is easier than this. This is the kind of thing we start looking at in the next couple sections.

Problems from the text

Lecture 7: 1-5

Lecture 8 Gram-Schmidt Orthogonalisation

As simple as the Gram Schmidt algorithm is, it turns out that it is not so good to do on computers. The error that is introduced by rounding tends to blow up. There is a small variation of Gram Schmidt that better controls this error.

The modified Gram-Schmidt algorithm

To orthogonalise basis $\{a_1, \ldots, a_n\}$ of a subspace of $\mathbb{C}^m$ to an orthogonal basis $\{q_1, \ldots, q_n\}$ we compute for each i

$$q_i' = a_i - (q_1^* a_i)q_1 - (q_2^* a_i)q_2 - \cdots - (q_{i-1}^* a_i)q_{i-1}.$$

Now for $j \leq i$ let $q_i^{(1)} = a_i - (q_1^* a_i)q_1$ we have already removed its q_1 component so we really don't have to remove it again in computing $q_2^* a_i$:

$$q_2^* q_i^{(1)} = q_2^*(a_i - (q_1^* a_i)q_1) = q_2^* a_i.$$

If there is no error, these are exactly the same. But if we have error, then because $q_i^{(1)}$ doesn't contain the error in the q_1 direction, the error in computing $q_2^* q_i^{(1)}$ tends to be smaller than that in computing $q_2^* a_i$.

So it is better to compute successive updates

$$\begin{aligned} q_i^{(j)} &= a_i - (q_1^* q_i^{(1)})q_1 - (q_2^* q_i^{(2)})q_2 - \cdots - (q_{j-1}^* q_i^{(j-1)})q_{j-1} \\ &= q_i^{(j-1)} - (q_{j-1}^* q_i^{(j-1)})q_{j-1} \end{aligned}$$

of q_i' for $j = 1, \ldots, i$ to finally get $q_i' = q_i^{(i)}$.

Algorithmically, after computing q_i, we remove its component from the vectors $a_{i+1}, \ldots, a_n$ at the same time.

Algorithm 8.1. *(Modified Gram Schmidt)*

Input: Independent vectors $\{a_1, \ldots, a_n\}$.

Output: Pairwise orthogonal unit vectors $\{q_1, \ldots, q_n\}$.

Algorithm:

i. For $i = 1, \ldots, n$ initialise $q_i^{(0)} := a_i$.

ii. For $j = 1, \ldots, n$ do

   (a) $q_j := q_j^{(j-1)} / \|q_j^{(j-1)}\|$.

   (b) For $i = j+1, \ldots, n$ compute $q_i^j := q_i^{j-1} - \text{proj}_{q_j}(q_i^{j-1})$.

Operation Count

We've talked about not wanting to do Gaussian Elimination too much. There are two reasons an algorithm can be bad for big matrices. It can be unstable– susceptible to error, but also its computation cost can be high.

The currency of computation is a *flop*– a floating point operation such as addition or multiplication of two numbers (floating point numbers of some fixed precision).

The dot product of two m-vectors has m products and $m-1$ additions. This is at most $2m-1$ flops. We usually write this as $\sim 2m$ flops where $f(m,n) \sim g(m,n)$ means

$$\lim_{m,n\to\infty} \frac{f(m,n)}{g(m,n)} = 1,$$

or as $O(m)$ flops, meaning that it takes at most cm flops for some fixed c. Pure mathematicians are happy with even a huge c here, but for us it will usually be something modest– if it gets bigger than 100 we will make a note of it.

Problem

Show that computing a product $A_{\ell\times m}B_{m\times n}$ costs $\sim 2\ell mn$ flops.

What is the cost of Gram-Schmidt for n vectors in $\mathbb{C}^m$. In computing $q_i' = a_i - \sum_{j=1}^{i-1}(q_j^* a_i)q_j$ it take $\sim 3m$ operations to compute $(q_j^* a_i)q_j$ and m more to subtract it, so this takes $\sim i \cdot 4m$ flops. We have to do this for $i = 1, \ldots, n$ so step (i) takes $\sim 4m\sum_i = 1^n i = \sim 2mn^2$ flops. Step (i), the normalising, takes $\sim n \cdot 3m$ flops, which when added to $\sim 2mn^2$ doesn't change it. So the algorithm costs $\sim 2mn^2$ flops. Gram-Schmidt is $O(mn^2)$.

Problem

Show that the modified Gram-Schmidt algorithm also costs $O(2mn^2)$ flops.

Problems from the text

Lecture 8: 1

Midterm

Lecture 9 Matlab

I won't do this section. If you have not already, it is probably a really good idea to try some of these algorithms in some computing language, or some computer algebra system. Matlab is good if you have it, Octave is a free program that has very similar code to Matlab– usually Matlab code runs fine in it if the code doesn't use too many libraries. Python is nice of course, usually with the numpy and sympy packages, and Sage is a nice open source project that is a lot like a wrapper for Python with all sorts of libraries that are nice for doing symbolic math.

Lecture 10 Householder Triangulisation

Nov 06

Householder Triangularisation is another method- different from Gram-Schmidt (GS) for getting the QR-decomposition of a matrix. We said that modfied GS was better than GS for error, the Householder method will be even better.

We get a QR-decompostion of $QR = A = [\,a_1|\ldots|a_n\,]$ by multiplying A on the left by several orthogonal matrices $Q_1, \ldots, Q_n$ such that

$$Q_n Q_{n-1} \ldots Q_1 A =: R$$

is upper triangular.

The first step in triangulating $A^{(0)} := A$ is to find a unitary (orthogonal) matrix $Q_1 = H_1$ such that

$$A^{(1)} = Q_1 A = \left[\begin{array}{c|c} \pm\|a_1\| & ? \\ \hline 0 & \\ 0 & ? \\ 0 & \end{array}\right]$$

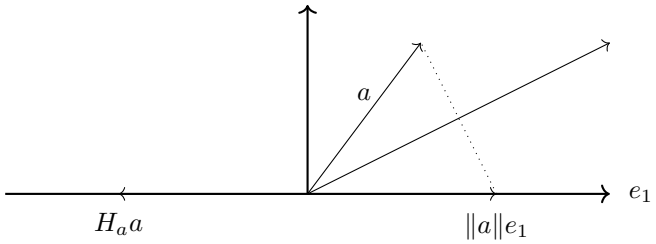
Notation:
Direct sum of matrices
$A \oplus B = \begin{bmatrix} A & 0 \\ 0 & B \end{bmatrix}$

where we have eliminated under the first entry. If we can find a unitary matrix H_a for any vector a such that $H_a a$ is this column, then we are happy. Taking $Q_i = I_i \oplus H_{b_i}$ where b_i is vector of the bottom $n-i$ entries of the ith column of $A^{(i-1)}$, we get that

$$A^{(2)} := Q_2 A^{(1)} = Q_2 Q_1 A = \left[\begin{array}{c|c|c} 1 & 0\ 0 & 0 \\ \hline 0 & & \\ 0 & H_{b_2} & \\ 0 & & \end{array}\right] \left[\begin{array}{c|c} \|a_1\| & ? \\ \hline 0 & \\ 0 & ? \\ 0 & \end{array}\right] = \left[\begin{array}{cc|c} \|a_1\| & ? & ? \\ 0 & \|b_2\| & \\ \hline & & \\ 0 & 0 & ? \\ & & \end{array}\right].$$

Continuing in this way, we will get upper diagonal $R = A^{(n)}$, and as the product of unitary matrices is unitary $Q = (Q_n Q_{n-1} \ldots Q_1)^*$ is the unitary matrix that we want.

The matrix H_a we will use (or $-H_a$) is called the *Householder reflection* of a– an orthogonal matrix such that $-H_a a = \|a\| e_1$. This reflects a onto the x axis through the hyperspace that bisects the angle between a and e_1.



Indeed, for vector a let $v = a + \|a\| e_1$. Let

$$H = H_a := I - 2P_v = I - 2\frac{vv^*}{v^*v}.$$

Problem

Prove the following.

Lemma 10.1. *Let $H = H_a$ be the Householder reflection for a vector a. The following are true.*

i. $Ha = -\|a\|e_1$

ii. H *is invertible*

iii. H *is Hermitian*

iv. H *is orthogonal*

Properties ii. - iv. are also true of $Q = I \oplus H_a$.

If a has a negative first coordinate, then we want to flip the picture, so use $v = a - \|a\|e_1$. Where $\text{sign}_1(a)$ is the sign of the first coordinate of a we thus use

$$v = a + \text{sign}_1(a)\|a\|e_1.$$

One might observe that using $-H_a$ in place of H_a will accomplish the same goal: $-H_a a$ is also a column with a non-zero element on the top of a bunch of zeros. And this is true, but $H_a a$ is further from a than $-H_a a$ is, and this adds conditioning to the algorithm. (We talk about conditioning soon.)

The algorithm

We have the Householder algorithm theoretically. The text introduces matrix slicing notation from Matlab. When A is a matrix $A_{i:i',j:j'}$ is the $(i'-i)\times(j'-j)$ submatrix consisting of the rows in the range $[i, i']$ and columns in the range $[j, j']$. So our vector b_i is $A^{(i-1)}_{i:m,i}$. As one does in a computer program, we will not make new matrices at each step, but will update our matrix. Instead of $A^{(2)} = Q_1 A^{(1)}$ we will say $A := Q_1 A$. This changes the matrix A.

Algorithm 10.2. *(Householder Triangulisation)*

Input: An $m \times n$ matrix A

Output: Upper triangular $\hat{R}$ such that $A = \hat{Q}\hat{R}$.

Algorithm: For $i = 1, \ldots, n$:

i. $b := A_{i,(i+1):m}$.

ii. $v := b + \text{sign}_1(\text{b})\|\text{b}\|\text{e}_1$ (e_1 is in $\mathbb{R}^{m-i}$ to match b)

iii. $u := v/\|v\|$

iv. $A := [I_i \oplus (I - 2uu^*)]A$

Return: $\hat{R} = A$

Problem

Go through the algorithm. Compute the cost of steps i. - iv. in flops. Verify that step iv. is most expensive.

Notice that we haven't returned Q, or the $Q_i = I_i \oplus (I - 2u^*u)$ that we used to compute them. We tend not to compute them, as they are a lot of zeros, and writing them out takes time. When i is big we are really only updating a small $m - i \times n - i$ matrix in the bottom left corner of A– each column c of this matrix, an $(m-i)$-vector, has $2uu^*c$ removed from it:

$$c := c - 2u(u^*c)$$

This costs $O((m-i)n)$ flops rather than the $O(m^2n)$ it takes to multiply Q_iA. Summing this for $i = 1, \ldots, n$ it takes (if $m \geq n$) order

$$\sum_{i=1}^{n} n(m-i) = n(mn - \sum_{i=1}^{n} i) = n(mn - n^2/2) = (m - n/2)n^2$$

flops. This is better than the $O(m^2n^2)$ it would take if we computed all of the Q_i.

Problem

If $m < n$ then the sum is $i = 1, \ldots, m$. Compare the two versions of the algorithm in this case.

So what if we want Q? We used it to solve $Ax = b$ when $A = QR$. We multiplied Q^*b to get a vector y, so it is useful.

In computing $R = Q^*A$ we certainly saw how to multiply a vector by Q^*, we did it for each column of A. We need only the u_i from that algorithm. Keeping them increases the space we need, but not the time; and it takes less space than holding Q.

Algorithm 10.3. *(Computing Q^*b)*

Input: The vectors b and $u_1, \ldots, u_n$ from the previous algorithm.

Output: The vector $y = Q^*b$.

Algorithm: For $i = 1, \ldots, n$:

i. Acting on only the bottom $m - i$ entries $b_{i:m}$ of b, replace $b_{i:m}$ with $b_{i:m} - 2u_i(u_i^*b_{i:m})$.

Return: $y = b$

This algorithm takes order

$$\sum_{i=1}^{n} m - i = mn - n^2/2$$

flops, compared to the $O(mn)$ it would even take to compute Q^*b if we had Q^*. Applying this to the columns of I_m, we can get Q^* (or Q) in $O(mn^2 - n^3/2)$ flops. (The text is a bit more precise giving $\sim 2mn^2 - \frac{2}{3}n^3$.)

Problem: *

Take a random full rank 4×4 matrix A and triangulate it via the Householder algorithm. Keep track of the vectors u_i and use them to compute Q^*b for some vector b.

Problem

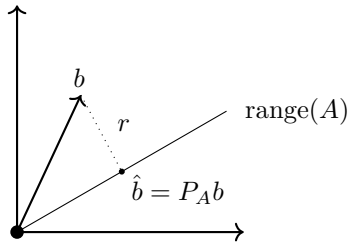
Show that to compute Qb for some vector b we can use the algorithm for computing Q^*b doing the operations for $i = n, \ldots, 1$ rather than $i = 1, \ldots, n$. (Hint: Remember that the Q_i are Hermitian.)

Problems from the text

Lecture 10: 1

Lecture 11 Least Squares Problems

Often the matrix equation $Ax = b$ one wants to solve has no solution– b is not in the range of A. In this case one settles for the vector $\hat{x}$ that minimises the length of the *residual* $\|r\|_2 := \|A\hat{x} - b\|_2$. As minimising the Euclidean norm is minimising a sum of squares, this $\hat{x}$ is the solution to the *least squares problem* for $Ax = b$. The closest vector to b in the range of A is its projection $\hat{b} = P_A b$, and so $A\hat{x} = \hat{b}$. When A has independent columns, this solution is unique.



An application

The text gives an interesting application of this to fitting polynomials to data. Say you have m data points $(x_1, y_1), \ldots, (x_m, y_m)$, and would like to interpolate a polynomial relation $y = f(x)$ from them. We know that there is a unique polynomial of degree $m-1$ that fits these points. Indeed it is the solution $f(x) = \sum_{i=0}^{m-1} c_i x^i$ to the matrix equation

$$\begin{bmatrix} 1 & x_1 & x_1^2 & & x_1^m \\ 1 & x_2 & x_2^2 & \cdots & x_2^m \\ & \vdots & & \ddots & \\ 1 & x_m & x_m^2 & & x_m^m \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{m-1} \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}.$$

The problem with this is that to fit these points exactly, some of the c_i are going to be really crazy. The polynomial we get fits the point, but doesn't look like it will be representative of data from which the points are sampled.

If we try to use a lower degree polynomial, the above equation (with some columns removed) usually doesn't have a solution. But an approximate solution, a least squares approximation, tends to give a much better representation of the data. The data can be fit exactly to a polynomial of degree m but that polynomial can look wild, a least squares approximation with a lower degree polynomial can give a much clearer picture of the data. See the text for some nice pictures in a particular example.

Pseudo Inverse

How do we find the least squares approximation $\hat{x}$ to a solution of $Ax = b$? We know how to find $\hat{b}$ such that $A\hat{x} = \hat{b}$ when the columns of A are independent, and so A^*A is invertible:

$$\hat{b} = P_A b = A(A^*A)^{-1}A^*b.$$

To compute $\hat{x}$ we can solve

$$Ax = \hat{b}.$$

If A itself is invertible then this has solution

$$x = A^{-1}A(A^*A)^{-1}A^*b = (A^*A)^{-1}A^*b$$

but this doesn't actually need that A is invertible. Just set

$$\hat{x} = (A^*A)^{-1}A^*b.$$

Mutliplying by A tells us that $A\hat{x} = \hat{b}$.

The matrix

$$A^+ := (A^*A)^{-1}A^*$$

is the *pseudo-inverse* of A.

Problem

When $n < m$ what is the best order (in terms of running time) to do the products to compute $\hat{x}$ and $\hat{b}$? (Using Guassian Elimination to invert the $n \times n$ matrix A^*A takes time $\sim n^3 + n^2$.) In particular, will you want to compute A^+?

The Normal Equation

To compute the pseudo-inverse, we have to compute A^*A and its inverse. But we don't really need to compute it. As long as it exists the equation $\hat{x} = (A^*A)^{-1}A^*b$ is equivalent to the *normal equation*

$$A^*A\hat{x} = A^*b,$$

so it is enough to solve this equation. We can do so by computing $y = A^*b$, computing A^*A and then solving the $n \times n$ system $(A^*A)x = y$. The Choelsky Factorisation from Lecture 23 does this, but does so taking advantage of the fact that A^*A is symmetric.

For now, we look at how a QR-decompositon or an SVD-decomposition help in solving the normal equation without computing A^*A.

Least Squares via QR

From a reduced QR-decomposition $A = \hat{Q}\hat{R}$ of A, we know that A and $\hat{Q}$ have the same column-space, so

$$\hat{b} = P_A b = P_Q b = \hat{Q}\hat{Q}^*b$$

and $\hat{Q}\hat{R}\hat{x} = A\hat{x} = \hat{b} = \hat{Q}\hat{Q}^*b$ so

$$\hat{R}\hat{x} = \hat{Q}^*b.$$

The psuedo inverse $A^+ = \hat{R}^{-1}\hat{Q}^*$ becomes much faster to compute, we are inverting now the upper triangular matrix $\hat{R}$ rather than computing and inverting A^*A. But still, to find $\hat{x}$ if we don't need $\hat{b}$ we would.

Algorithm 11.1. *(Least squares $\hat{x}$ via QR)*

i. Compute reduced QR-decomposition $A = \hat{Q}\hat{R}$.

ii. Compute $y = \hat{Q}^* b$.

iii. Solve the upper-triangular system $\hat{R}x = y$.

Least Squares via SVD

From a reduced SVD-decomposition $A = \hat{U}\hat{\Sigma}V^*$, we again have $P_A = P_{\hat{U}}$ so

$$\hat{b} = \hat{U}\hat{U}^* b$$

and $\hat{U}\hat{\Sigma}V^*\hat{x} = A\hat{x} = \hat{b} = \hat{U}\hat{U}^* b$ so

$$\hat{\Sigma}V^*\hat{x} = \hat{U}^* b.$$

Algorithm 11.2. *(Least squares $\hat{x}$ via SVD)*

i. Compute reduced QR-decomposition $A = \hat{U}\hat{\Sigma}V^*$.

ii. Compute $y = \hat{U}^* b$.

iii. Solve the diagonal system $\hat{\Sigma}w = y$.

iv. Set $\hat{x} = Vw$.

The text does a detailed accounting of the running times of these algorithms, but the are all $O(mn^2)$ for $m > n$. Some will be better in some cases and others better in other cases. What will recommend one over the other though will be its accuracy when run on a computer– its stability. It is time to look at this.

Problem: *

Pick a matrix A of reasonable dimensions, 8×3 say and a vector b not in its column space. Using a computer, solve the least squares problem by computing the pseudo inverse (by 'hand'– don't use a function pseudoinverse(A), but compute A^* then A^*A, then its inverse...). Find a reduced QR-decomposition, again by 'hand', using any of the algorithms we've seen in class. Find a reduced SVD-decompositoin using a builtin function (we haven't really done it yet.) Then solve least squares again using the above algorithms.

Problems from the text

Lecture 11: 1

Part III: Conditioning and Stability

Lecture 12 Conditioning and Condition numbers

Nov 13

When we put matrices into a computer, especially after we run a couple of operations, we introduce error from the round-off of the numbers. How does this error effect singular vectors of matrix A or the the solution to an equation $Ax = b$? It depends a lot on how we compute them.

We can think of a problem such as solving $Ax = b$ as a function, in this case a function $f : \mathbb{C}^{mn} \to \mathbb{C}^n$ such that for matrix A, $f(A) = A^{-1}b$ is the solution to $Ax = b$. Or it could be a function of b, or of A and b. We will write $f : X \to Y$ to omit the necessity to address dimension, but you can think of these both as complex (or real) vector spaces.

The condition of the problem $f(x)$ is then a measure of how much of a change in $f(x)$ we get from a small change in the x. This sounds like derivatives, and we can use calculus to formalise this, but lets try to simplfy it a bit.

For a problem $f : X \to Y$ and $x \in X$, let δx be the perturbation of x by some small error and let

$$\delta f(x) = f(x + \delta x) - f(x)$$

be the corresponding change in $f(x)$. The *absolute condition number* of the problem at x is

$$\hat{\kappa}(x) = \lim_{\delta \to 0} \sup_{\|\delta x\| \le \delta} \frac{\|\delta f(x)\|}{\|\delta x\|}.$$

This is the max of the linear directional derivitives of $f(x)$ at x, and is shown in a multivariate calculus class to be the norm $\|J\|$ of the *Jacobian* matrix J whose entries are the partials derivatives $\partial f_i / \partial x_j$ of the componets of f.

More often, we are interested in the (relative) condition number of $f(x)$ with normalised error

$$\kappa(x) = \lim_{\delta \to 0} \sup_{\|\delta x\| \le \delta} \frac{\|\delta f(x)\|/\|f(x)\|}{\|\delta x\|/\|x\|} = \frac{\|J\|}{\|f(x)\|/\|x\|}.$$

We generally write just $\sup_{\delta x}$ for $\lim_{\delta \to 0} \sup_{\|\delta x\| \le \delta}$, understanding by use of 'δ' that these are taken infinitesimally.

Example 12.1. Let $f(x) = \sqrt{x}$. The Jacobian is the 1×1 matrix $[\frac{\mathrm{d}}{\mathrm{d}x}\sqrt{x}] = [1/2\sqrt{x}]$. So

$$\kappa = \frac{\|J\|}{\|f(x)\|/\|x\|} = \frac{1/2\sqrt{x}}{\sqrt{x}/x} = 1/2$$

A problem is *well-conditioned* if κ is small for all instances of the problem, say $\kappa = 1$ or 100; it is *ill-conditioned* if κ is big, say 10^5, for some instance. The problem of finding the eigenvalues of a matrix is known to be ill-conditioned. (See the book for an example.)

Often we don't use the Jacobian, but directly use

$$\kappa(x) = \sup_{\delta x} \frac{\|\delta f(x)\|/\|f(x)\|}{\|\delta x\|/\|x\|}.$$

Example 12.2. The condition of the problem of matrix-vector multiplication $f(x) = Ax$ depends on A. We have that

$$\delta f(x) = f(x+\delta) - f(x) = A(x+\delta x) - Ax = A\delta x.$$

$$\kappa(x) = \sup_{\delta x} \frac{\|A\delta x\|/\|Ax\|}{\|\delta x\|/\|x\|} = \sup_{\delta x} \frac{\|A\delta x\|}{\|\delta x\|}\frac{\|x\|}{\|Ax\|} = \|A\|\frac{\|x\|}{\|Ax\|} \leq \|A\|\|A^{-1}\|,$$

where we only have the last inequality if A is inverible; and for x that maximises $\frac{\|x\|}{\|Ax\|}$ it is equality.

This

$$\kappa(A) := \|A\|\|A^{-1}\| = \sigma_1/\sigma_m$$

is used so much it is called the *condition number of* A. It shows up in several places, indeed. as solving $Ax = b$ for fixed A is just the condition number of the problem $f(b) = x = A^{-1}b$, we have that this also has condition number $\|A\|\|A^{-1}\| = \kappa(A)$ for the worst instances b.

Practice

Find the condition number for the problem $f(A) = x$ of solving $Ax = b$, for fixed b.

Problems from the text

Lecture 12: 1

Lecture 13 Floating point numbers

Fixing the number t of usable bits allotted to represent a number on computer, let $F = F_t$ be the set of numbers that we can represent with t bits; these are called the floating points numbers, or floats. We will ignore the actual value of t but it is usually 24 or 53 depending on whether we are on a 32-bit or 64-bit achitecture. Any real number r has a *floating point representation* $\mathrm{fl}(r)$ such that

$$\|\mathrm{fl}(r)/r\| \leq 1 \pm 2^{-t} =: 1 \pm \varepsilon_{\mathrm{mach}}.$$

This $\varepsilon_{\mathrm{mach}} := 2^{-t}$ is called *machine epsilon*, and $B_{\mathrm{mach}} = \{\delta \in X \mid |\delta| < \varepsilon_{\mathrm{mach}}\}$ is its epsilon ball in X, which will usually be determined by context.

We start with an assumption on the error of our basic operations. For any operation $* \in \{+, -, \times, \div\}$ let $\circledast$ be the operation on the set F of fixed point numbers (for fixed t).

We will assume that the computer can compute these functions as accurately as possible over F– we assume that for any $x, y \in F$,

$$x \circledast y = \mathrm{fl}(x * y). \tag{1}$$

As we can write $x, y \in F$ as $\mathrm{fl}(x)$ and $\mathrm{fl}(y)$, this identity, the *fundamental axiom of floating point arithmetic* can be viewed as a type of commutation of operations $\mathrm{fl}(x) \circledast \mathrm{fl}(y) = \mathrm{fl}(x * y)$ when x and y are floats.

With the definition of $\varepsilon_{\mathrm{mach}}$, this gives the useful corollary that for any basic operation $*$ and all $x, y \in F$ there is $\delta \in B_{\mathrm{mach}}$ such that

$$x \circledast y = (x * y)(1 + \delta). \tag{2}$$

For the problems, IEEE sinlge precision means take $t = 24$ and IEEE double precision means take $t = 53$.

Problems from the text

Lecture 13: 1, 2, 3 (Obviously, use a computer for 3.)

Lecture 14 Stability

Where $f : X \to Y$ is a problem, let $\tilde{f} : X \to Y$ be a floating point algorithm for the problem– one that uses only floating point numbers.

An algorithm $\tilde{f}$ for a problem f is *accurate* if for all $x \in X$

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|} = O(\varepsilon_{\text{mach}}).$$

Note

This big-O notation means that as $\varepsilon_{\text{mach}}$ goes to 0, the relative error in computed $\tilde{f}(x)$ goes to 0– it is less than $C\varepsilon_{\text{mach}}$ for some fixed C. This is a theoretical measure of the accuracy of the algorithm. In practice, our hardware (or the architecture) fixes $\varepsilon_{\text{mach}}$. If C is big, approaching $1/\varepsilon_{\text{mach}}$ then the accurate algorithm is useless. But this is not the algorithm's fault. It is waiting there for better computers with even smaller $\varepsilon_{\text{mach}}$, and then it will be accurate in practice.
Generally in this section though, the constant C of a $O(\varepsilon_{\text{mach}})$ is modest, less than 1000, for values of $\varepsilon_{\text{mach}}$ that we are interested in.

We can assume the two constants for $O(\varepsilon_{\text{mach}})$ in the def of stability are the same– the second one is 'for some', which is forgiving.

Accuracy is impossible for an ill-conditioned problem; this is no fault of the algorithm. But for well-conditioned problems, some algorithms will be accurate.

An algorithm $\tilde{f}$ for a problem f is *stable* if for each $x \in X$

$$\frac{\|\tilde{f}(x) - f(\tilde{x})\|}{\|f(\tilde{x})\|} = O(\varepsilon_{\text{mach}}) \qquad \text{for some } \tilde{x} \text{ with } \qquad \frac{\|\tilde{x} - x\|}{\|x\|} = O(\varepsilon_{\text{mach}}).$$

It is *backward stable* if for each $x \in X$

$$\tilde{f}(x) = f(\tilde{x}) \qquad \text{for some } \tilde{x} \text{ with } \qquad \frac{\|\tilde{x} - x\|}{\|x\|} = O(\varepsilon_{\text{mach}}).$$

The text gives a nice way to think of this– a problem is stable if it gives nearly the right answer to nearly the right question, it is backwards stable if it gives the right answer to nearly the right question.

Stable algorithms for well-conditioned problems are generally pretty accurate, but proving accuracy for them can be difficult. Showing that an algorithm is backwards stable is usually harder than showing that it is stable, but once you've shown it, the following gives you accuracy for free.

Theorem 14.1. *Let $\tilde{f}$ be a backwards stable algorithm for a problem $f : X \to Y$ with condition number κ. For any x in X*

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|} = O(\kappa\varepsilon_{\text{mach}}).$$

Problem

Prove this theorem.

Note

- All of these definitions talk of a functions with a vector x as input, but many of the functions we care about will have several vectors or matrices as input. This is okay, as these can all be viewed as a vector: the input x of the problem of compuing the inner product of u and v can be viewed as the vector $x = u|v$ we get by concatenating u and v. No matter which norm we are using, we get that $\|u|v\| \leq \|u\| + \|v\|$, so if we can say that $\|u\|, \|v\| \in O(\varepsilon_{\text{mach}})$ we get that $\|x\| \in O(\varepsilon_{\text{mach}})$.
- The norm that we are using is not important to the definition of accuracy or stability. Indeed, we know that for any two norms $\|\cdot\|$ and $\|\cdot\|_*$ of $\mathbb{R}^n$ there are constants c_1 and c_2 such that for all $x \in \mathbb{R}^n$,

$$\|x\|c_1 \leq \|x\|_* \leq \|x\|c_2.$$

The fundamental theorem of fixed point arithmetic asserts that the basic operations of addition, subtraction, multiplication, and division all have backwards stable algorithms.

Example 14.2. Where $f(a, b) = a - b$ is subtraction, we show that $\tilde{f}(a, b) = \text{fl}(a) \ominus \text{fl}(b)$ is backwards stable.

By the definition of $\varepsilon_{\text{mach}}$, there are $\delta_a, \delta_b \in B_{\text{mach}}$ such that $\text{fl}(a) = a(1 + \delta_a)$ and $\text{fl}(b) = b(1 + \delta_b)$, and by (2) there is $\varepsilon \in B_{\text{mach}}$ such that $\text{fl}(a) \ominus \text{fl}(b) = (\text{fl}(a) - \text{fl}(b))(1 + \varepsilon)$. So taking $\tilde{a} = a(1+\delta_a)(1+\varepsilon)$ and $\tilde{b} = b(1+\delta_b)(1+\varepsilon)$, we get $|a-\tilde{a}| < 1+3\varepsilon_{\text{mach}}$ and $|b-\tilde{b}| < 1+3\varepsilon_{\text{mach}}$, so

$$\frac{\|a - \tilde{a}\|}{\|a\|}, \frac{\|b - \tilde{b}\|}{\|b\|} < 3\varepsilon_{\text{mach}}.$$

Further, we have

$$\tilde{f}(a, b) = \text{fl}(a) \ominus \text{fl}(b) = (a(1 + \delta_a) - b(1 + \delta_b))(1 + \varepsilon) = \tilde{a} - \tilde{b} = f(\tilde{a}, \tilde{b}),$$

showing that $\tilde{f}$ is backward stable.

Problem

Show that the sum $\bigoplus a_i$ of several numbers is backwards stable. Show that floating point division is backwards stable.

Example 14.3. The obvious floating point inner product

$$\tilde{f}(a, b) = \bigoplus(a_i \otimes b_i) = (a_1 \otimes b_1) \oplus \cdots \oplus (a_n \otimes b_n)$$

of two vectors is backward stable. Indeed by (2), for each i there are $\varepsilon_i \in B_{\text{mach}}$ such that

$$a_i \otimes b_i = a_i b_i (1 + \varepsilon_i),$$

and there is $\delta \in B_{\text{mach}}$ such that

$$\bigoplus(a_i \otimes b_i) = (\sum a_i \otimes b_i)(1+\delta) = \sum a_i b_i (1+\varepsilon_i)(1+\delta).$$

So $\tilde{f}(a,b) = f(a,\tilde{b})$ where $\tilde{b}_i = b_i(1+\varepsilon_i)(1+\delta)$ for each i, giving $\|\tilde{b}_i - b_i\| \leq 3\varepsilon_{\text{mach}}\|b_i\|$.

Example 14.4. The obvious floating point outer-product is not backward stable. Indeed, the obvious floating point algorithm $\tilde{f}$ for the outer-product problem $f(uv) = uv^T$ would return a matrix $A = [a_{ij}]$ with $a_{ij} = u_i \otimes v_j$. This algorithm is stable, as A is less than $n^2\varepsilon_{\text{mach}}$ from uv^T. But A generally has high rank. We cannot write it as the rank one matrix $\tilde{u}\tilde{v}^T$ for any $\tilde{u}$ and $\tilde{v}$. So the algorithm is not backwards stable.

Problems from the text

Lecture 14:

Highlights of rest of Lectures 14-19

We will not go through the verifications of stability and backwards stability of our main algorithms. This is done in the next 5 lectures of the text, but it is a bit of a slog. Here are some of the highlights.

One, is that the Householder algorithm for computing the QR-decomposition of a matrix is backwards stable. What does this mean?

The QR-decomposition $A = QR$ we get from Gauss Schmidt or Householder, if there is no error, will give essentially the same Q and R. But in floating point, either algorithm will give some error, and this error will be different for the two algorithms. When we find a QR-decomposition $A = QR$ via the Householder algorithm we get $A \approx \tilde{Q}\tilde{R}$. Backwards stability means that there is some $\tilde{A}$ close to A such that $\tilde{A} = \tilde{Q}\tilde{R}$. In the text they show by example that $\tilde{Q}$ and $\tilde{R}$ can be relativley far from Q and R. So if our goal it really to find Q and R, this is not so good. But our goal is often to use the decompostion to compute the solution to $Ax = b$, and for this they can be quite useful– their acuracy in computing this depends on the relative conditon number of A. $\tilde{A}$ is close to A, so the solution to $Ax = b$ is close to $\tilde{A}x = b$.

Lecture 18 looks at the condition of the least squares problem for $Ax = b$. While we have been saying that a backwards stable algorithm for a well-conditioned problem is accurate, this is parameterised. The condition of the least squares problem depends on A and b, and its sensitivity to changes in A and b can be different. They show that problems of finding $\hat{b}$ and $\hat{x}$ are better conditioned the closer b is to orthogonal to range(A).

Lecture 19 looks at the stability of the least squares problem. Solving it by the Householder QR-decomposition is backwards stable. Gram-Schmidt, even the modified version, is unstable, but there are ways to make it more stable. One is via pivoting, but the text does not explain this, but the main idea is to reorder columns of A and Q as we go along in the modified algorithm, so that at each step we choose the norm-largest column remaining as the next one to put into our basis. They mentioned another method using techniques similar to those used in the Householder algorithm, but I couldn't understand what they were saying.

This analysis was all for $Ax = b$ where A has rank n. If it is has dependent columns, then things are a little different. Only the SVD method works fine in this setting. The Gram Schmidt algorithm modification we mentioned earlier when A has dependent columns can be viewed as column pivoting, and this can be used to deal with this problem. One can pivot columns in Householder too, to deal with low rank.

Along they way they address the stability of several algorithms.

- Unstable algorithms: Using the characteristic polynomial to find eigenvalues. Least square by computing A^*A.
- Stable: Floating point outer-product. SVD-decompositions (we haven't done yet).
- Backwards stable: Floating point algorithms of the Basic operations, including inner products. Householder QR-decompositions. Least squares via Householder QR. Least square via modified Gram Schmidt QR-decompitions. Back-substitution (solving $Rx = b$ for upper triangular R by Gaussian Elimination). Solving $Ax = b$ by the QR-decomposition. Full rank least squares by SVD.

Part IV: Systems of Equations

Lecture 20 Gaussian Elimination

Recall the idea of Gaussian elimination from Lecture 1.

$$\begin{bmatrix} 2 & -2 & 2 \\ 2 & 1 & 1 \\ 1 & 3 & -1 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 2 & -2 & 2 \\ 0 & 3 & -1 \\ 0 & 4 & -2 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 2 & -2 & 2 \\ 0 & 3 & -1 \\ 0 & 0 & -\frac{3}{4} \end{bmatrix}.$$

We will mostly only use this on square matrices, so will assume that A is $m \times m$, and our goal will not be fully reducing A to I, but to go to an upper triangular matrix U. We saw that each of these basic operations can done by multiplying on the left by a lower-trianular matrix.

$$\underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -\frac{4}{3} & 1 \end{bmatrix}}_{L_2} \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ -\frac{1}{2} & 0 & 1 \end{bmatrix}}_{L_1} \begin{bmatrix} 2 & -2 & 2 \\ 2 & 1 & 1 \\ 1 & 3 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -\frac{4}{3} & 1 \end{bmatrix} \begin{bmatrix} 2 & -2 & 2 \\ 0 & 3 & -1 \\ 0 & 4 & -2 \end{bmatrix} = \underbrace{\begin{bmatrix} 2 & -2 & 2 \\ 0 & 3 & -1 \\ 0 & 0 & -\frac{3}{4} \end{bmatrix}}_{U}$$

Multiplying these L_i together we get lower-triangular L such that $LA = U$.

$$L = L_2 L_1 = \begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ \frac{5}{6} & -\frac{4}{3} & 1 \end{bmatrix}$$

Problems from the text

Lecture ??: